Name：Bowen Liu Student ID：1155254007

Agentic AI for Business and FinTech (FTEC5660)

PART 1: KYC system

1. System Architecture and Design Decisions

The CV verification system is designed as an autonomous Agentic AI application that automates the manual, time-consuming process of cross-referencing candidate resumes against public social media profiles.

Core Architecture:

- **Agent Orchestrator:** The system utilizes the LangGraph framework (specifically the create_react_agent function) to build a ReAct (Reasoning and Acting) agent. This allows the Large Language Model (LLM) to iteratively think about the problem, select appropriate tools, observe the results, and formulate the next steps.
- **Tool Integration (MCP Server):** The agent connects to a provided Model Context Protocol (MCP) server. This server grants the agent controlled access to external social graph data , specifically LinkedIn and Facebook tools.
- **Asynchronous Execution:** To handle network latency and tool invocations efficiently without blocking the main execution thread, the agent is implemented using asynchronous asynchronous calls (ainvoke).

Key Design Decisions:

1. **Dual-Platform Strategy:** Relying on a single platform can lead to false negatives if a candidate is inactive there. The architecture mandates querying both Facebook and LinkedIn to gather a comprehensive digital footprint.

2. **Pipeline-in-Prompt:** Instead of hardcoding a rigid Python script for each step, the system uses an advanced "Pipeline-in-Prompt" design. The system prompt enforces a strict sequence (Extract -> Search -> Retrieve -> Evaluate), allowing the LLM to autonomously handle exceptions (e.g., trying a fuzzy search if an exact match fails) while adhering to strict business logic.

2. Agent Workflow and Tool Usage Strategy

The agent operates on a structured, multi-stage workflow designed to minimize LLM hallucinations and maximize identity resolution accuracy. Below are the core workflow steps and key code explanations:

Stage 1: Information Extraction & Entity Resolution

- The agent begins by parsing the raw CV text to extract the candidate's name, location, current company, and education.
- Tool Strategy: It uses search_linkedin_people and search_facebook_users. To avoid "entity resolution failure" (picking the wrong person with the same name), the system instructs the agent to read the headline and location from the search list to identify the correct profile before extracting the integer id.

Stage 2: Deep Data Retrieval & Agent Execution Logic

- Tool Strategy: Once the correct integer IDs are identified, the agent calls get_linkedin_profile and get_facebook_profile to retrieve complete professional and personal backgrounds.

Core Invocation Code:

```
# Initialize Agent and bind LLM with MCP tools
agent_executor = create_react_agent(llm, tools, prompt=system_prompt)

# Construct the verification request for the current CV and execute asynchronously
user_message = f"Please verify this resume using the 3-stage chain process:\n\n{cv_text}"
response = await agent_executor.ainvoke({"messages": [HumanMessage(content=user_message)]})
final_answer_raw = response["messages"][-1].content
```

Code Explanation: The system instantiates the agent using create_react_agent. Through the asyncio-based ainvoke method, the LLM autonomously determines the sequence of tool calls (e.g., calling search tools first, acquiring IDs, then calling detail retrieval tools) without blocking the program. The final reasoning result after multiple tool interactions is returned to final_answer_raw.

Stage 3: Best-Match Baseline & Discrepancy Detection

- The agent compares the CV against both retrieved profiles to determine which platform provides a closer, more reliable baseline identity.
- It then detects discrepancies between the CV claims and the selected social media data.

Result Extraction Code:

```
# Use regex to extract structured JSON data from the Agent's free-text response
json_match = re.search(r'```json\n(.*?)\n```', final_answer, re.DOTALL)
if json_match:
    result_dict = json.loads(json_match.group(1))
    extracted_score = float(result_dict.get("score", 0.5))
    reasoning = result_dict.get("reasoning", "No reasoning provided")
```

Code Explanation: Due to the inherent randomness in LLM outputs, the system forces the capture of JSON blocks matching a specific format via regular expressions. This ensures that the code can safely and consistently extract the score and underlying reasoning computed by the Agent for the final automated evaluation loop.

Stage 4: "Smart Tolerance" Scoring

- The agent generates a final authenticity score between 0.0 and 1.0.

- **Tool Strategy:** A "Smart Tolerance" rule is applied to the logic. CVs are static documents; therefore, if a candidate claims to be currently employed on their CV but the LinkedIn profile shows is_current: false, the agent treats this as normal job-seeking behavior rather than immediate fraud, provided the company and core dates match. However, fundamental lies regarding highest degrees or completely fabricated current roles are heavily penalized.

3. Sample Verification Results

The system was tested against 5 sample CVs containing potential discrepancies. The agent successfully analyzed all files, achieving a 100% accuracy rate against the evaluation ground truth.

```
========================================
Currently checking [1/5]: CV_1.pdf
========================================
Extracted score: 0.7
Reasoning: The LinkedIn profile (ID 9) provided the better match,

========================================
Currently checking [2/5]: CV_2.pdf
========================================
Extracted score: 1.0
Reasoning: The LinkedIn profile (ID 47) provided an excellent mat

========================================
Currently checking [3/5]: CV_3.pdf
========================================
Extracted score: 0.8
Reasoning: The Facebook profile provided the better match, as no

========================================
Currently checking [4/5]: CV_4.pdf
========================================
Extracted score: 0.2
Reasoning: The LinkedIn profile (ID 765) was considered the best

========================================
Currently checking [5/5]: CV_5.pdf
========================================
Extracted score: 0.4
Reasoning: The LinkedIn profile (ID 95) provided the best baselir


All CVs have been verified.
List of scores given by Agent: [0.7, 1.0, 0.8, 0.2, 0.4]
```

{'decisions': [1, 1, 1, 0, 0], 'correct': 5, 'total': 5, 'final_score': 1.0}